

Understanding & *Attacking* Large Language Models

A working reference for security engineers: how LLMs are built and served, where their attack surface lives, and how adversary behaviour maps onto the MITRE ATLAS and ATT&CK matrices — grounded in the current published data, not paraphrased memory.

Scope LLM internals → threat modelling → real tooling

ATLAS **baseline** v2026.09 (16 tactics • 208 techniques)

ATT&CK **baseline** Enterprise v19 (Apr 2026)

SECTION 01

Introduction to Large Language Models

Before you can attack or defend an LLM, you need an accurate mechanical picture of what it is: a probabilistic next-token predictor, trained at scale, wrapped in an application. Almost every attack in this document exploits one of those three layers.

1.1 Definition

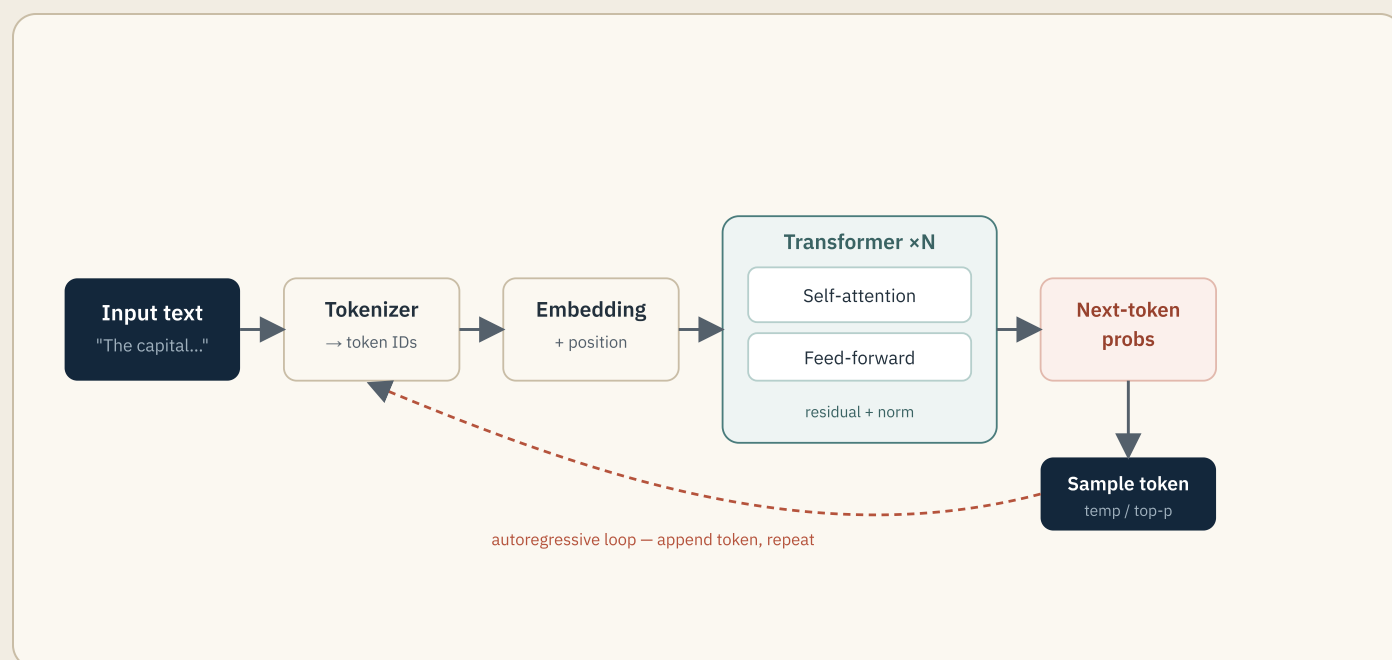
A **Large Language Model (LLM)** is a neural network — in practice, a *Transformer* — trained on very large text corpora to model the probability distribution of the next unit of text (a *token*) given the text that came before it. "Large" refers to two dimensions that scale together: the number of learned parameters (billions to trillions) and the size of the training corpus (hundreds of billions to trillions of tokens).

Two properties matter for security:

- **It is generative and probabilistic.** The model does not retrieve a stored answer; it samples a plausible continuation. The same input can yield different outputs, and confidently-worded output can be entirely fabricated (a *hallucination*). Attackers weaponise both — hallucinations become squattable package names, and non-determinism frustrates naive signature detection.
- **It cannot separate instructions from data.** A classic program has a control plane (code) and a data plane (input) that are architecturally distinct. An LLM receives a single undifferentiated stream of tokens. System prompt, developer instructions, user input, and retrieved documents all arrive as text and are weighed by the same attention mechanism. *This single fact is the root cause of prompt injection* — the defining vulnerability class of the LLM era.

1.2 How LLMs work

The pipeline from raw text to generated output has a small number of stages. Understanding each one tells you where an adversary can insert themselves.



Inference pipeline. Text is split into tokens, mapped to vectors, passed through N Transformer blocks, and turned into a probability distribution over the vocabulary. One token is sampled, appended, and the loop repeats.

Tokenization

Text is split into *tokens* — sub-word units (e.g. `token`, `##ization`) using a scheme such as Byte-Pair Encoding. Tokenization is exploitable: unusual Unicode, invisible characters, or

encoding tricks (Base64, ROT13) can produce token sequences that slip past keyword-based guardrails while still meaning something to the model.

Embedding & positional encoding

Each token ID becomes a high-dimensional vector (an *embedding*) that captures learned meaning; positional information is added so the model knows token order. Semantically similar tokens sit close together in this space — the property that makes embedding-based retrieval (and RAG) possible, and that also lets an attacker craft inputs that are "near" a target concept.

The Transformer block & self-attention

The core innovation (Vaswani et al., 2017, "*Attention Is All You Need*") is **self-attention**: for every token, the model computes how much every other token should influence it, in parallel. Stacking many attention + feed-forward blocks lets the model build a rich contextual representation. Because attention weighs *all* tokens in the context window against each other, an injected instruction buried in a retrieved document can pull the model's behaviour just as strongly as the developer's system prompt.

Output distribution & sampling

The final layer emits a probability (via *softmax*) over the whole vocabulary for the next token. A *decoding* strategy then picks one — greedily, or by sampling controlled by **temperature** and **top-p**. Higher temperature means more randomness. This stage is why identical prompts give varying answers, and why "just block the bad output string" is not a durable control.

■ SECURITY READ-THROUGH

Map the stages to attack surface: **tokenizer** → obfuscation/encoding bypass; **context window** → prompt injection & instruction/data confusion; **training data** → poisoning & backdoors; **output layer** → data leakage, insecure rendering, unsafe tool calls. Every later section returns to these four.

1.3 Importance & impact

LLMs matter to security not because they are novel, but because of *where they are being placed*: as reasoning layers wired directly into email, code repositories, ticketing systems,

customer support, SOC tooling, and — most consequentially — **autonomous agents** that can call tools, browse, and execute actions on a user's behalf.

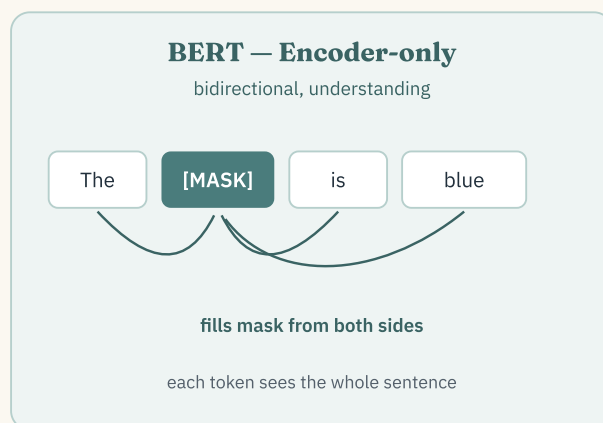
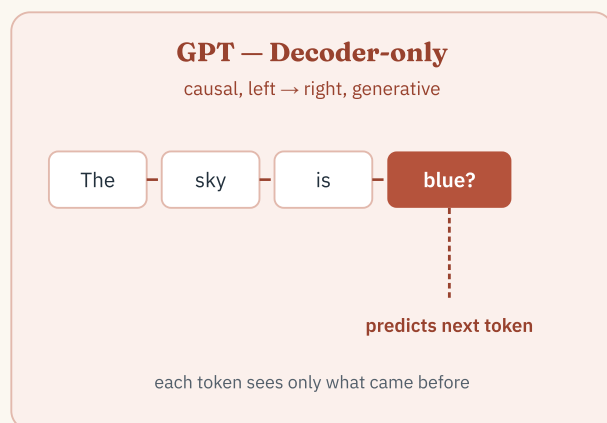
- **New attack surface, old consequences.** A prompt injection is novel; the outcome — arbitrary code execution, data exfiltration, fraudulent transactions — is not. The LLM becomes a *confused deputy*: a trusted component that an attacker persuades to misuse its own privileges.
- **Trust inversion.** Organisations grant agents broad access on the assumption the model is loyal. Once the model can be steered by untrusted input (a web page, an email, a calendar invite), that access becomes the attacker's access.
- **Scale for the offence too.** The same generative capability that helps defenders writes fluent phishing at volume, generates polymorphic malware, and — as of 2026 — drives semi-autonomous intrusion campaigns. Section 6 covers the criminal tooling built around exactly this.

This is why frameworks like **MITRE ATLAS** exist: to give this new-but-familiar threat landscape a shared vocabulary, which Section 5 works through tactic by tactic.

SECTION 02

Understanding LLMs: Two Architectures

"Transformer" is one idea with two dominant families. Knowing which family a system uses tells you what it can be made to do — and therefore how you would attack it.



Decoder-only models (GPT) generate one token at a time, attending only to the left context — ideal for open-ended generation. Encoder-only models (BERT) attend in both directions to build a representation — ideal for understanding and classification.

2.1 GPT — Generative Pre-trained Transformer

GPT is a **decoder-only**, *autoregressive* architecture. It is trained with a single objective — predict the next token — over enormous unlabelled corpora, then adapted to follow instructions (Section 3). "Causal" (or masked) self-attention means each position can attend only to earlier positions, which is what makes coherent left-to-right generation possible.

- **Pre-trained** — general language competence learned once, at great cost, from raw text.
- **Generative** — produces novel text, code, and structured output rather than labelling existing text.
- **Transformer** — the attention-based backbone from Section 1.

GPT-family models (and their open counterparts — LLaMA, Mistral, Falcon, GPT-J, and so on) are what people mean by "an LLM" in the chat/agent sense. They are the target of prompt injection, jailbreaks, system-prompt extraction, and tool-abuse — the generative-AI-specific techniques that dominate Section 5. Notably, **GPT-J 6B**, an open model released by EleutherAI in 2021, is the base that the original WormGPT was built on (Section 6).

2.2 BERT — Bidirectional Encoder Representations from Transformers

BERT (Google, 2018) is an **encoder-only** architecture built for *understanding* rather than generation. Its key training objective is the *Masked Language Model*: random tokens are hidden and the model predicts them using context from *both* directions. This bidirectional context yields rich representations well-suited to classification, named-entity recognition, sentiment analysis, and — relevant to security — powering the embedding and retrieval components inside larger systems.

BERT is less a chatbot and more a workhorse for text understanding embedded in pipelines. In an attack context it appears on both sides: as a defensive classifier (e.g. an ML phishing/malware detector an attacker wants to evade), and as a component an attacker may try to poison or fingerprint.

■ WHY THE DISTINCTION MATTERS FOR ATTACKERS

A **generative (GPT-style)** target invites input-side manipulation — you steer what it *produces* and what tools it *invokes*. A **discriminative (BERT-style)** target invites evasion and poisoning — you manipulate inputs to flip a *classification*, or corrupt training data to plant a backdoor. ATLAS reflects this split explicitly through its Predictive AI VS Generative AI VS Agentic AI platform tags on each technique.

SECTION 03

Training & Augmenting LLMs

A model's capabilities are shaped in three ways: what it learns during pre-training, how it is fine-tuned afterward, and what external knowledge is fed to it at run time. Each is a distinct point of attack.

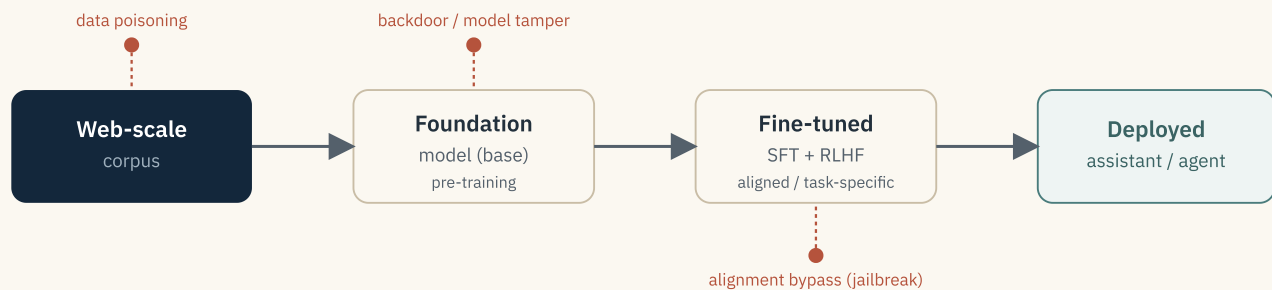
3.1 Foundational vs fine-tuned models

A **foundation (base) model** is the product of expensive, general-purpose *pre-training* on a broad corpus. It has broad competence but is not yet aligned to a task or to human

preferences — a raw base model will happily complete text without regard for helpfulness or safety.

A **fine-tuned model** takes that foundation and continues training on a narrower, curated dataset to specialise it. Two important sub-types:

- **Supervised fine-tuning (SFT) / instruction tuning** — teaches the model to follow instructions and adopt a helpful assistant format.
- **Alignment tuning (RLHF / preference optimisation)** — shapes the model toward human-preferred, safer responses. *Guardrails and refusals largely live here*. Jailbreaks (ATLAS [AML.T0054](#)) are attempts to override precisely this layer.



The training lifecycle and where adversaries insert themselves: poisoning the pre-training corpus, tampering with or backdooring the model artifact, or bypassing the alignment layer at inference time.

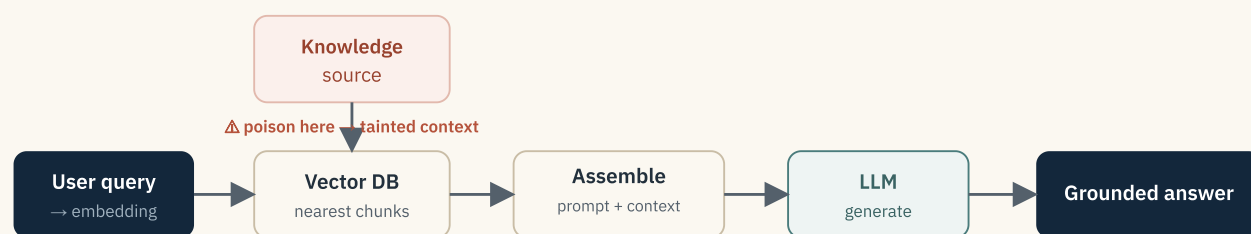
■ ATTACK SURFACE ACROSS THE LIFECYCLE

- **Training-time:** data poisoning ([AML.T0020](#)), model backdoors / trojans, and publishing poisoned datasets or models to the supply chain ([AML.T0115](#)). A poisoned base model contaminates every downstream fine-tune.
- **Artifact-time:** a distributed model file can carry malware in an unsafe serialization format, or be corrupted to defeat a scanner while still executing ([AML.T0076](#)). Real cases include malicious models on Hugging Face and the compromised PyTorch dependency chain.
- **Inference-time:** jailbreaks and prompt injection attack the aligned behaviour directly, without touching weights.

3.2 Retrieval-Augmented Generation (RAG)

A fine-tuned model still only knows what was in its training data, frozen at a cutoff. **RAG** closes that gap by fetching relevant external documents at query time and inserting them into the prompt as context, so the model can answer using current or private knowledge it never memorised.

The flow: the user query is embedded into a vector; a *vector database* returns the nearest-matching document chunks; those chunks are concatenated with the query and the system prompt; the assembled prompt goes to the LLM; the LLM answers grounded in the retrieved text.



retrieved documents enter the prompt as trusted text — but they are just tokens
RAG Poisoning (AML.T0070) · False RAG Entry Injection (AML.T0071) · RAG Credential Harvesting (AML.T0082)

RAG architecture. The knowledge source and vector store are new, high-value attack surface: content an adversary can get indexed becomes context the model treats as authoritative.

■ WHY RAG IS A SECURITY PIVOT

RAG dissolves the boundary between "the model's knowledge" and "attacker-supplied text." Anything an adversary can plant where the RAG system indexes — a wiki page, a shared drive, a public web page, an email — can surface as retrieved context and carry hidden instructions (indirect prompt injection) or false facts. This is why ATLAS has an entire cluster of RAG-specific techniques, and why the *Morris II* self-replicating worm and *Slack AI / M365 Copilot* exfiltration cases all pivot through retrieval. Treat every indexed source as untrusted input.

Use Cases of LLMs

The point of cataloguing use cases here is not the capabilities themselves but the *trust* and *access* each one is granted. Where an LLM is wired, and what it is allowed to touch, defines the blast radius when it is subverted.

4.1 Text Generation

The canonical capability: producing fluent novel text — drafting, summarising, translating, code generation, and content creation. Generation is what makes LLMs productive and what makes them dangerous in adversarial hands.

■ OFFENSIVE MISUSE

Fluent generation at scale powers phishing and business-email-compromise lures ([AML.T0052.000](#) — Spearphishing via Social Engineering LLM), malware and exploit code ([AML.T0102](#) — Generate Malicious Commands), disinformation, and deepfake text/media ([AML.T0088](#)). The criminal tools in Section 6 exist almost entirely to remove the guardrails that would otherwise refuse these requests.

4.2 Text Understanding

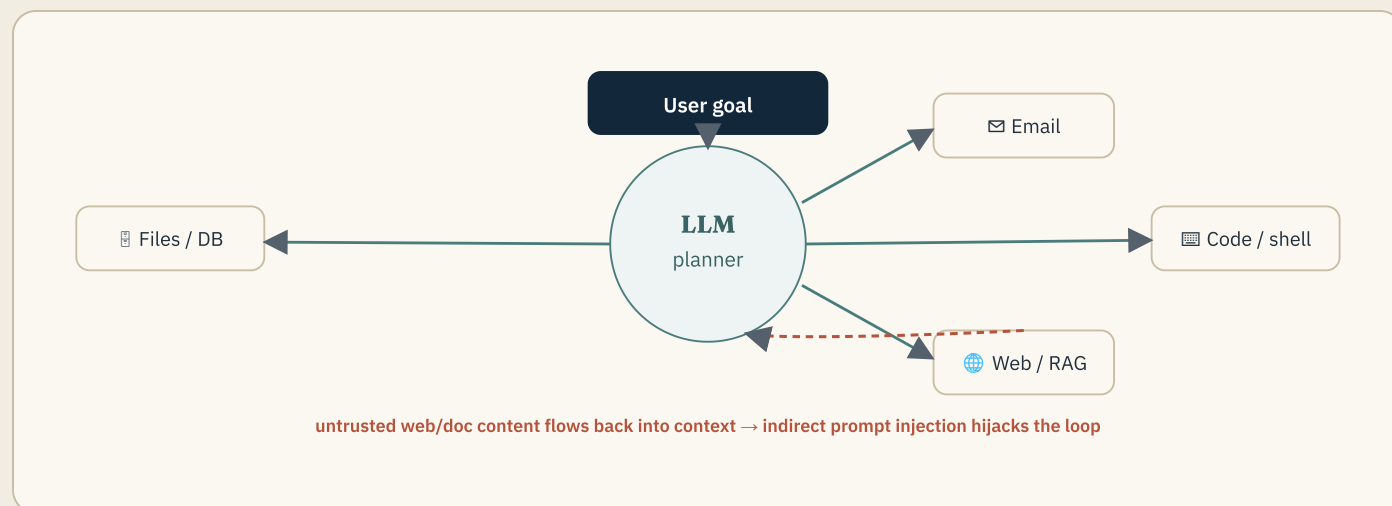
Classification, extraction, sentiment, semantic search, and content moderation — the discriminative tasks BERT-style models excel at. This is heavily used *defensively*: ML-based malware detectors, phishing-page classifiers, and spam filters are themselves models.

■ OFFENSIVE MISUSE

When the "understanding" model is a defence, the attacker's goal is **evasion** ([AML.T0015](#) — Evade AI Model) or **poisoning** its training data. ATLAS case studies include evading a deep-learning malware C2 detector, bypassing Cylance's AI malware detection, ProofPoint evasion, and attempted evasion of an ML phishing-webpage detector. When the understanding model is the attacker's *own* reconnaissance tool, it triages stolen data or fingerprints targets at scale.

4.3 Conversational AI & Agents

Multi-turn assistants that maintain context — and, increasingly, **agents** that plan and act: calling tools, querying databases, running code, browsing the web, sending email, and operating other software (including via the Model Context Protocol, MCP). This is the highest-risk deployment pattern because the model's decisions become *actions in the world*.



The agentic loop. The model reads a goal, plans, calls tools, observes results, and repeats. The danger: tool *outputs* (a fetched web page, a retrieved email) re-enter the context as trusted text and can carry instructions that redirect every subsequent action.

■ THE CONFUSED-DEPUTY PROBLEM, MADE CONCRETE

An agent inherits the privileges of the identity it runs as. If untrusted content can steer it (indirect injection, poisoned RAG, a crafted email or calendar invite), the attacker inherits those privileges without ever authenticating. ATLAS's newest tactics and mitigations — *AI Agent Authority Expansion Controls* ([AML.M0037](#)), *Human In-the-Loop for AI Agent Actions* ([AML.M0029](#)), *Restrict AI Agent Tool Invocation on Untrusted Data* ([AML.M0030](#)) — exist specifically to bound this. Section 5's agentic techniques (tool poisoning, tool invocation, data destruction via tools) all live in this loop.

Attack Tactics & Techniques

A tactic is an adversary's *goal* ("why"); a technique is *how* they achieve it. MITRE organises both into a matrix so defenders can reason about coverage. There are two matrices you must not confuse.

5.1 MITRE ATT&CK

MITRE ATT&CK (Adversarial Tactics, Techniques & Common Knowledge) is the globally-used knowledge base of adversary behaviour against *conventional* IT — endpoints, networks, cloud, ICS. Tactics are ordered roughly by attack progression, from Reconnaissance through Impact. Every technique carries an ID like **T1566** (Phishing).

ATT&CK is a *living* framework. Its most significant structural change in years shipped as **ATT&CK v19 on 28 April 2026**: the long-overloaded *Defense Evasion* tactic (**TA0005**) was split into **Stealth** (which keeps **TA0005** — blending into legitimate behaviour) and **Defense Impairment** (**TA0112** — actively disabling controls). Enterprise ATT&CK v19 comprises 15 tactics, 222 techniques, and 475 sub-techniques. If your detections still map to the old unitary "Defense Evasion," they now describe a narrower world and need remapping.

5.2 MITRE ATLAS matrix

ATLAS (Adversarial Threat Landscape for Artificial-Intelligence Systems) is ATT&CK's counterpart for AI. It reuses ATT&CK's Tactic → Technique → Mitigation → Case-study structure but focuses on attacks against AI-enabled systems, abuse of AI capabilities, and harmful autonomous behaviour. IDs use the **AML.** prefix — tactics **AML.TA####**, techniques **AML.T####**, mitigations **AML.M####**, case studies **AML.CS####**.

Most ATLAS tactics are inherited from ATT&CK; **two are AI-native** — *AI Model Access* and *AI Attack Staging* (renamed *AI Attack Adaptation* in the 2026.08 release, reflecting agents that adapt an attack at runtime). The framework has grown rapidly with agentic-AI threats: the release used as the baseline throughout this section, **ATLAS v2026.09 (15 September 2026)**, contains **16 tactics, 208 techniques, 40 mitigations and 73 case studies** — of which 23 are documented real-world incidents rather than red-team exercises.

Dimension	MITRE ATT&CK (Enterprise v19)	MITRE ATLAS (v2026.09)
Target	Conventional IT: endpoints, network, cloud, ICS	AI/ML systems: models, data, pipelines, agents
ID prefix	T##### / TA#####	AML.T##### / AML.TA#####
Tactics	15	16 (14 shared + 2 AI-native)
AI-native tactics	—	AI Model Access · AI Attack Staging / Adaptation
Signature techniques	Phishing, LSASS dumping, DLL side-loading	Prompt injection, jailbreak, RAG poisoning, model extraction, agent tool poisoning
Relationship	Complementary — ATLAS techniques frequently chain into ATT&CK ones once the attacker is on a host. Many ATLAS techniques carry an Enterprise platform tag and reference their ATT&CK parent.	

Frameworks: MITRE ATLAS atlas.mitre.org (data v2026.09, Apache-2.0, © 2021–2026 The MITRE Corporation) and MITRE ATT&CK Enterprise v19. Counts and technique details below are taken directly from the published ATLAS dataset, not summarised from memory.

5.3 The ATLAS matrix at a glance

Tactics read left-to-right as an operation progresses (though real attacks loop and skip). Techniques shown in **rust** are generative-/agentic-AI-specific — the LLM-era additions. The two AI-native tactic columns have a rust header. Scroll horizontally to see all sixteen.

■ Generative / agentic-AI technique ■ Shared with ATT&CK ■ AI-native tactic

Reconnaissance AML.TA0002	Resource Development AML.TA0003	AI Attack Adaptation AML.TA0001	Initial Access AML.TA0004
Search Open Technical Databases	Acquire Public AI Artifacts	Create Proxy AI Model	AI Supply Chain Compromise
Search Open AI Vulnerability Analysis	Acquire Infrastructure	Manipulate AI Model	Valid Accounts
Search Victim-Owned Websites	Obtain Capabilities	Verify Attack	Evade AI Model
Search Application Repositories	Develop Capabilities	Craft Adversarial Data	Exploit Public-Facing Application
Active Scanning	Establish Accounts	LLM Prompt Crafting	Phishing
	Publish Hallucinated Entities	Retrieval Content Crafting	Drive-by Compromise
	Stage Capabilities	Generate Deepfakes	

Gather RAG-Indexed Targets	Publish Poisoned AI Artifacts	Generate Malicious Commands	Prompt Infiltration via Public-Facing Application
Gather Victim Identity Information	Compromise Infrastructure	Autonomous Attack-Path Adaptation	Exploit Automated Artifact Processing Pipeline
Search Open Websites/Domains		Autonomous AI Agent Communication	Crafted AI Assistant Links
Autonomous Reconnaissance		Autonomous Attack Orchestration	Misconfigured or Publicly Exposed AI Services

All 16 tactics with their top-level techniques, taken verbatim from ATLAS v2026.09. Sub-techniques are omitted here for legibility and expanded in the per-tactic notes below.

5.4 The tactics, technique by technique

Each tactic below opens with the adversary goal, then lists its techniques with ATLAS IDs and platform tags (Gen = Generative AI, Agent = Agentic AI, Pred = Predictive AI, Ent = Enterprise). Descriptions are condensed from the ATLAS v2026.09 dataset. Two tactics from the full matrix — *Lateral Movement* and *Command & Control* — sit among these in the matrix above; both are largely shared with ATT&CK and are folded into the discussion where they connect.

AML.TA0002

01. Reconnaissance

Gather information about the target AI system to plan the operation. This is where an attacker learns what model family is in use, what its attack surface looks like, and what data feeds it — much of it from entirely public sources. The 2026 additions push reconnaissance into autonomy: an agent given only a goal can decide what to look for.

AML.T0000 **Search Open Technical Databases** Ent

Mine published research, pre-print repos, technical blogs and scan databases for details of a target model or its stack.

AML.T0001 **Search Open AI Vulnerability Analysis** Ent

Search public AI vulnerability write-ups and disclosures relevant to the target.

AML.T0003 **Search Victim-Owned Websites** Ent

Harvest AI-relevant details from the victim's own websites.

AML.T0004 **Search Application Repositories** Ent

Inspect public app repositories for AI code, configs and secrets.

AML.T0006 **Active Scanning** Gen·Agent·Pred·Ent

Actively probe hosts and APIs — enumerate hosted AI resources, query platform metadata APIs, scan for exposed AI infra, and probe agent trigger channels.

AML.T0064 **Gather RAG-Indexed Targets** Gen·Agent

Identify the data sources a RAG system indexes, so they can later be poisoned.

AML.T0087 **Gather Victim Identity Information** Ent

Collect victim identity data (names, emails, MFA config) for targeting.

AML.T0095 **Search Open Websites/Domains** Ent

Search public sites/domains, including code repositories, for attack-surface intel.

AML.T0116 **Autonomous Reconnaissance** Gen·Agent·Pred·Ent

Use an autonomous agent to drive reconnaissance from a goal or partial lead — deciding what to look for and how.

02. Resource Development

Establish the resources needed to attack: infrastructure, accounts, and — distinctively for AI — *artifacts*. This tactic includes publishing poisoned models and datasets, standing up AI-service proxies to launder frontier-model access, and “slopsquatting”: registering the fake packages an LLM is known to hallucinate.

AML.T0002 Acquire Public AI Artifacts Gen·Agent·Pred

Acquire public AI artifacts — datasets, models, or agent configurations — to study or repurpose.

AML.T0008 Acquire Infrastructure Ent

Stand up infrastructure: dev workspaces, hardware, domains, serverless, and AI-service proxies that resell frontier-model access.

AML.T0016 Obtain Capabilities Gen·Agent·Pred·Ent

Obtain off-the-shelf capabilities — attack implementations, tooling, generative-AI, exploits, and agent tools.

AML.T0017 Develop Capabilities Gen·Agent·Pred·Ent

Develop custom capabilities: adversarial attacks, autonomous exploit development, and malicious agent tools.

AML.T0021 Establish Accounts Ent

Register accounts with services for targeting, staging, or impersonation.

AML.T0060 Publish Hallucinated Entities Gen·Agent

Register a real entity (package, domain, email) matching a name an LLM reliably hallucinates — “slopsquatting.”

AML.T0079 Stage Capabilities Ent

Stage developed or obtained capabilities on infrastructure the adversary controls.

AML.T0115 Publish Poisoned AI Artifacts Gen·Agent·Pred

Publish poisoned datasets, models or agent tools through public distribution channels to compromise downstream systems.

AML.T0128 Compromise Infrastructure Ent

Compromise third-party infrastructure and repurpose it against the AI target.

AML.TA0004

03. Initial Access

Get a foothold in the AI system. Alongside classic vectors (valid accounts, exploiting public apps, phishing), the LLM era adds prompt-infiltration — planting malicious prompts where an AI will later ingest them — crafted assistant links, and simply finding the many AI services left publicly exposed. Note that *prompt injection itself* is frequently the initial-access vector.

AML.T0010 AI Supply Chain Compromise Gen·Agent·Pred

Compromise the AI supply chain — hardware, AI software, data, models, container registries, or agent tools.

AML.T0012 Valid Accounts Ent

Use valid stolen or default credentials to access the system.

AML.T0015 Evade AI Model Pred

Feed adversarial input crafted to be misclassified, gaining access by defeating an ML gate.

AML.T0049 Exploit Public-Facing Application Ent

Exploit a vulnerability in a public-facing application.

AML.T0052 Phishing Ent

Phishing — including LLM-authored spearphishing and deepfake-assisted lures.

AML.T0052.000 Spearphishing via Social Engineering LLM.

AML.T0052.001 Deepfake-Assisted Phishing.

AML.T0078 Drive-by Compromise Ent

Drive-by compromise: a visited page (or an agent fetching the web) delivers an injection or payload.

AML.T0093 Prompt Infiltration via Public-Facing Application Gen·Agent

(also Persistence) Seeded prompts that persist in a public-facing app to affect future AI interactions.

AML.T0119 Exploit Automated Artifact Processing Pipeline Ent

Submit an artifact that triggers an automated processing pipeline and exploits weak processing logic.

AML.T0131 Crafted AI Assistant Links Gen·Agent

Craft links that open an AI assistant with attacker-supplied input already loaded (e.g. via a “?q=” parameter).

AML.T0132 Misconfigured or Publicly Exposed AI Services Gen·Agent·Pred

Reach AI services left misconfigured or publicly exposed — missing auth, permissive access.

AML.TA0000

04. AI Model Access

AI-NATIVE TACTIC Gain some level of access to the model itself — the precondition for many AI-specific attacks. Access ranges from black-box (an inference API), through indirect (an AI-enabled product), to full white-box knowledge of weights and architecture. The level of access an attacker holds dictates which staging and exfiltration techniques become available.

AML.T0040 AI Model Inference API Access Gen·Agent·Pred

Legitimate inference-API access — a means to probe, stage, and extract.

AML.T0041 Physical Environment Access Gen·Agent·Pred

Access the physical environment where the model’s input data is collected.

AML.T0044 Full AI Model Access Gen·Agent·Pred

Full white-box access — architecture, parameters, ontology — enabling offline attack crafting.

AML.T0047 AI-Enabled Product or Service Gen·Agent·Pred

Indirect access via an AI-enabled product or service that may leak model detail in logs/metadata.

AML.TA0005

05. Execution

Run adversary-controlled behaviour — code, or model actions. This is the home of **LLM Prompt Injection (AML.T0051)**, the defining LLM technique, and of **AI Agent Tool Invocation**, where a subverted agent is made to call its own tools. In agentic systems, “execution” increasingly means the model taking real actions in the world.

AML.T0011 User Execution Gen·Agent·Pred·Ent

Rely on a user to load/run an unsafe artifact, malicious package, poisoned agent tool, or malicious link.

AML.T0050 Command and Scripting Interpreter Ent

Abuse a command/script interpreter to run code.

AML.T0051 LLM Prompt Injection Gen·Agent

LLM Prompt Injection — craft inputs that override the model’s instructions. Direct, indirect (via ingested data), or triggered by an environment event.

AML.T0051.000 Direct — the adversary injects as the user.

AML.T0051.001 Indirect — injection arrives via ingested data (docs, email, web, DB).

AML.T0051.002 Triggered — fires on a user action or environment event.

AML.T0053 AI Agent Tool Invocation Agent

AI Agent Tool Invocation — use agent access to call its tools (APIs, data, code execution).

AML.T0100 AI Agent Clickbait Agent

Bait a computer-using agent or AI browser with deceptive UI/content into clicking or navigating.

AML.T0103 Deploy AI Agent Agent

Deploy the adversary's own AI agent inside the victim environment to act on their behalf.

AML.TA0006

06. Persistence

Keep the foothold across restarts, resets, and credential changes. For AI this means poisoning the model or its training data, corrupting an agent's memory or configuration so changes outlive a single session, poisoning RAG stores, and self-replicating prompts that propagate on their own — the mechanism behind worm-style attacks like Morris II.

AML.T0018 Manipulate AI Model Gen·Agent·Pred

Manipulate the model — poison it, alter architecture, embed malware, or change prompt-construction logic.

AML.T0020 Training Data Poisoning Gen·Agent·Pred

Poison training data so learned behaviour serves the adversary.

AML.T0061 LLM Prompt Self-Replication Gen·Agent

A self-replicating prompt that induces the LLM to reproduce it in output — propagating to other LLMs (worm-like).

AML.T0070 RAG Poisoning Gen·Agent

RAG Poisoning — inject malicious content into RAG-indexed data so it surfaces in future results.

AML.T0080 AI Agent Context Poisoning Gen·Agent

AI Agent Context Poisoning — persistently alter an agent's memory or thread context to change its behaviour.

AML.T0080.000 Memory.

AML.T0080.001 Thread.

AML.T0081 **Modify AI Agent Configuration** Agent

Modify AI agent configuration (system prompt, knowledge sources, tools) so changes persist across sessions.

AML.T0099 **AI Agent Tool Data Poisoning** Agent

Poison data in a source an agent tool reads from (repos, DBs, knowledge bases).

AML.T0110 **AI Agent Tool Poisoning** Agent

AI Agent Tool Poisoning — corrupt a tool's definition, implementation, or runtime response (incl. MCP servers).

AML.T0110.000 Definition & Instructions.

AML.T0110.001 Implementation.

AML.T0110.002 Runtime Response.

AML.T0121 **AI Agent Environment Reconstruction** Ent

An autonomous agent reconstructs its environment (tools, creds, access) to survive a reset.

AML.T0125 **Create Account** Ent

Create an account to maintain access to the AI system or its infrastructure.

AML.TA0012

07. Privilege Escalation

Gain higher-level permissions. In an AI context the standout is the **LLM Jailbreak (AML.T0054)**: overriding the model's alignment escalates what it is willing to do, effectively raising the attacker's privilege over the model's behaviour. Container escape and valid-account abuse escalate on the underlying host.

AML.T0012 **Valid Accounts** Ent

Use valid stolen or default credentials to access the system.

AML.T0053 **AI Agent Tool Invocation** Agent

AI Agent Tool Invocation — use agent access to call its tools (APIs, data, code execution).

AML . T0054 LLM Jailbreak Gen · Agent

LLM Jailbreak — override the model’s safety/alignment to elicit withheld behaviour, escalating what it will do.

AML . T0105 Escape to Host Ent

Escape a container/VM to reach the host.

AML . TA0007

08. Defense Evasion

Avoid detection — by human reviewers, by LLM guardrails, and by AI-enabled security tools. This is ATLAS’s largest tactic, and it is rich with LLM-specific moves: obfuscating injections in invisible or encoded text, hiding triggers in one modality of a multimodal input, manipulating citations and chat history, false RAG entries, supply-chain rug pulls, and cloaking that shows benign content to humans but malicious content to AI clients.

AML . T0015 Evade AI Model Pred

Feed adversarial input crafted to be misclassified, gaining access by defeating an ML gate.

AML . T0054 LLM Jailbreak Gen · Agent

LLM Jailbreak — override the model’s safety/alignment to elicit withheld behaviour, escalating what it will do.

AML . T0067 LLM Trusted Output Components Manipulation Gen · Agent

Manipulate response components (language, links, citations) so the LLM appears trustworthy.

AML . T0068 LLM Prompt Obfuscation Gen · Agent

LLM Prompt Obfuscation — hide injections via tiny/invisible text, encoding (Base64/ROT13), or multimodal tricks.

AML.T0071 False RAG Entry Injection Gen·Agent

False RAG Entry Injection — smuggle a fake “retrieved document” inside a legitimate RAG entry.

AML.T0073 Impersonation Ent

Impersonate a trusted person/org to manipulate a target.

AML.T0074 Masquerading Ent

Masquerade — manipulate an artifact’s name/location to look benign to users and tools.

AML.T0076 Corrupt AI Model Gen·Agent·Pred

Corrupt a malicious model file so a scanner fails to deserialize it, while it still executes first.

AML.T0081 Modify AI Agent Configuration Agent

Modify AI agent configuration (system prompt, knowledge sources, tools) so changes persist across sessions.

AML.T0092 Manipulate User LLM Chat History Gen·Agent

Manipulate the user’s LLM chat history to cover tracks.

AML.T0094 Delay Execution of LLM Instructions Gen·Agent

Delay execution — instructions that fire on a future event/keyword to evade inspection.

AML.T0097 Virtualization/Sandbox Evasion Ent

Virtualization/sandbox evasion.

AML.T0107 Exploitation for Defense Evasion Ent

Exploit a vulnerability specifically to bypass a security feature.

AML.T0109 AI Supply Chain Rug Pull Gen·Agent·Pred

AI Supply Chain Rug Pull — publish a legitimate component, gain adoption, then push a malicious update.

AML.T0111 AI Supply Chain Reputation Inflation Gen·Agent·Pred

Inflate a malicious component's reputation with fabricated trust signals to drive adoption.

AML.T0123 Obfuscated Files or Information Pred·Ent

Obfuscate files/data by encryption or encoding to evade defenses, including AI defenders.

AML.T0129 Triggers in Multimodal Inputs Gen·Agent

Hide triggers in one modality of a multimodal input (e.g. text-in-image) unseen by single-modality filters.

AML.T0134 AI Targeted Cloaking Agent

AI Targeted Cloaking — serve malicious content to AI clients but benign content to humans/crawlers.

AML.TA0013

09. Credential Access

Steal credentials. The AI-specific twist: credentials leak *through* the AI. They get inadvertently indexed into RAG stores and retrieved, sit in agent configurations, or are harvested by an agent's own tools reaching into SharePoint, Drive, or GitHub — the model becomes the credential-exfiltration path.

AML.T0055 Unsecured Credentials Ent

Find insecurely stored credentials (plaintext files, env vars, keys).

AML.T0082 RAG Credential Harvesting Gen·Agent

RAG Credential Harvesting — pull credentials that were inadvertently indexed into a RAG store.

AML.T0083 Credentials from AI Agent Configuration Agent

Read credentials (API keys, tokens) out of an AI agent's configuration.

AML.T0090 OS Credential Dumping Ent

OS credential dumping from caches, memory, or stores.

AML.T0098 AI Agent Tool Credential Harvesting Agent

Use agent tool access to harvest credentials from connected sources (SharePoint, Drive, GitHub).

AML.T0106 Exploitation for Credential Access Ent

Exploit a software vulnerability to obtain credentials.

AML.T0113 Steal Web Session Cookie Ent

Steal a web session cookie to authenticate to an AI-enabled system.

AML.TA0008

10. Discovery

Map the AI environment from the inside. Beyond classic enterprise discovery, attackers fingerprint the model family, extract the system prompt and its special keywords, enumerate an agent's tools and activation triggers, and — crucially — discover the model's reliable hallucinations to feed the resource-development stage.

AML.T0007 Discover AI Artifacts Gen·Agent·Pred

Discover AI artifacts on the system — training stack, data systems, registries, model ZOOS.

AML.T0013 Discover AI Model Ontology Pred

Discover the model's ontology (its output classes).

AML.T0014 Discover AI Model Family Gen·Agent·Pred

Identify the general model family from docs or crafted probes.

AML.T0062 Discover LLM Hallucinations Gen·Agent

Discover LLM Hallucinations — prompt the model to find fake packages/URLs/orgs to later register.

AML.T0063 Discover AI Model Outputs Gen·Agent·Pred

Discover model outputs (class scores) exposed in logs or API responses.

AML.T0069 Discover LLM System Information Gen·Agent

Discover LLM system information — the system prompt, special-character sets, or capability keywords.

AML.T0075 Enterprise Resource Discovery Ent

Enterprise resource discovery (accounts, systems, storage, cloud objects).

AML.T0084 Discover AI Agent Configuration Agent

Discover an agent's configuration — embedded knowledge, tool definitions, activation triggers, call chains.

AML.T0089 Enterprise Environment Discovery Ent

Enterprise environment discovery (configs, connections, trust, defensive tooling).

AML.T0133 Discover AI Agent Runtime Capabilities Agent

Probe an agent at runtime to surface its tools, parameters, reach and permissions — without its config.

AML.TA0009

11. Collection

Gather the data and artifacts the operation is after. Models, datasets, and telemetry are collected for exfiltration or for offline attack staging; agentic tools and RAG databases become bulk-collection channels, often fully automated with no human selecting each item.

AML.T0035 AI Artifact Collection Gen·Agent·Pred

Collect AI artifacts (models, datasets, telemetry) for exfiltration or staging.

AML.T0036 Data from Information Repositories Ent

Data from information repositories.

AML.T0037 Data from Local System Ent

Data from the local system.

AML.T0085 Data from AI Services Gen·Agent

Data from AI services — pull proprietary info via RAG databases or agent tools.

AML.T0126 Automated Collection Ent

Automated collection via scripts, CLIs, or agent tools — no human picking each item.

AML.T0127 Data Staged Ent

Stage collected data centrally before exfiltration.

AML.TA0001

12. AI Attack Staging / Adaptation

AI-NATIVE TACTIC Turn capabilities and knowledge into a ready-to-fire attack, tailored to the target. This is where adversarial examples are crafted and *verified* against a proxy or the live API, prompts and retrieval content are engineered, deepfakes and malicious commands are generated, and — in the 2026.08 rename to *AI Attack Adaptation* — autonomous agents build, revise, coordinate, and communicate attack paths at runtime.

AML.T0005 Create Proxy AI Model Gen·Agent·Pred

Create a proxy model to simulate the target offline (train from artifacts, replicate, or use a pre-trained one).

AML.T0018 Manipulate AI Model Gen·Agent·Pred

Manipulate the model — poison it, alter architecture, embed malware, or change prompt-construction logic.

AML.T0042 Verify Attack Gen·Agent·Pred

Verify the attack works via inference API or an offline copy before deploying it.

AML.T0043 Craft Adversarial Data Pred

Craft adversarial data — white-box/black-box optimization, transfer, manual, or backdoor triggers.

AML.T0043.004 Insert Backdoor Trigger.

AML.T0065 LLM Prompt Crafting Gen·Agent

LLM Prompt Crafting — iteratively engineer prompts that reliably bypass defenses.

AML.T0066 Retrieval Content Crafting Gen·Agent

Retrieval Content Crafting — author content designed to be retrieved and influence the user.

AML.T0088 Generate Deepfakes Pred·Ent

Generate deepfakes — synthetic media for impersonation or fraud.

AML.T0102 Generate Malicious Commands Ent

Generate malicious commands dynamically from natural language — shifting signatures.

AML.T0117 Autonomous Attack-Path Adaptation Gen·Agent·Pred·Ent

Autonomous Attack-Path Adaptation — an agent builds and revises its own attack path toward a goal.

AML.T0118 Autonomous AI Agent Communication Agent·Ent

Autonomous AI Agent Communication — agents exchange tasking, discoveries and results (shared artifacts or direct).

AML.T0124 Autonomous Attack Orchestration Gen·Agent·Pred·Ent

Autonomous Attack Orchestration — one system coordinates multiple agents toward a shared objective.

AML.TA0010

13. Exfiltration

Steal data and model IP. AI opens novel channels: model extraction and inversion through the inference API, prompting the model to leak its training/connected data, inducing output that *renders* and beacons data to an attacker server (image/link exfil), and encoding stolen

data into an agent tool call that looks like legitimate work — the pattern behind the Slack AI, EchoLeak, and Copilot exfiltration cases.

AML.T0024 Exfiltration via AI Inference API Gen·Agent·Pred

Exfiltrate via the inference API — membership inference, model inversion, or full model extraction.

AML.T0024.000 Infer Training Data Membership.

AML.T0024.001 Invert AI Model.

AML.T0024.002 Extract AI Model — clone a private model by querying it.

AML.T0025 Exfiltration via Cyber Means Ent

Exfiltrate via conventional cyber means.

AML.T0056 Extract LLM System Prompt Gen·Agent

Extract the LLM system prompt (valuable IP and an attack aid).

AML.T0057 LLM Data Leakage Gen·Agent

LLM Data Leakage — prompt the model to reveal training data, connected data, or other users' data.

AML.T0077 LLM Response Rendering Gen·Agent

LLM Response Rendering — induce output that renders (image/link) and beacons private data to the attacker's server.

AML.T0086 Exfiltration via AI Agent Tool Invocation Agent

Exfiltration via AI Agent Tool Invocation — encode data into a tool call's parameters (email, doc, request).

AML.TA0011

14. Impact

Manipulate, interrupt, erode trust in, or destroy the AI system and its data. Includes denial-of-service against expensive AI compute, cost harvesting to run up the victim's bill, eroding model/dataset integrity to destroy confidence, biasing an assistant's recommendations,

and — for agents with write access — outright data destruction via tool invocation. The External Harms technique captures financial, reputational, societal, and user harm.

AML.T0015 Evade AI Model Pred

Feed adversarial input crafted to be misclassified, gaining access by defeating an ML gate.

AML.T0029 Denial of AI Service Gen·Agent·Pred

Denial of AI Service — flood or craft heavy-compute inputs to degrade/shut down the service.

AML.T0031 Erode AI Model Integrity Gen·Agent·Pred

Erode AI Model Integrity — degrade performance over time to erode trust and waste resources.

AML.T0034 Cost Harvesting Gen·Agent·Pred

Cost Harvesting — drive up the victim’s AI bill via excessive, resource-intensive, or agentic queries.

AML.T0034.002 Agentic Resource Consumption.

AML.T0046 Spamming AI System with Chaff Data Gen·Agent·Pred

Spam the system with chaff data to bury signal and waste analyst time.

AML.T0048 External Harms Gen·Agent·Pred·Ent

External Harms — financial, reputational, societal, user harm, and AI IP theft.

AML.T0048.004 AI Intellectual Property Theft.

AML.T0059 Erode Dataset Integrity Gen·Agent·Pred

Erode dataset integrity to reduce its usefulness and trust.

AML.T0101 Data Destruction via AI Agent Tool Invocation Agent

Data Destruction via AI Agent Tool Invocation — use a tool with write/delete power to wipe data.

AML.T0112 Machine Compromise Gen·Agent·Pred

Machine Compromise — exploit AI components to gain code execution and persist on the host.

AML.T0130 AI Agent Response Biasing Gen·Agent

AI Agent Response Biasing — bias an assistant toward attacker-chosen sources/content.

SECTION 06

Real-World LLM Attack Tools

In mid-2023, a wave of “dark GPTs” appeared on underground forums and Telegram — LLMs stripped of guardrails and sold to criminals. They are the practical embodiment of Section 4’s offensive misuse. Treat vendor claims with scepticism: several were credible, uncensored models; others were likely jailbroken-prompt wrappers or outright scams. What follows is drawn from the original threat-intel reporting.

■ HOW TO READ THESE

All three are “**uncensored assistant**” products: the selling point is the *removal* of the alignment layer (Section 3.1) so the model will write phishing, malware, and fraud content on demand. In ATLAS terms they are **AML.T0016.002** (Obtain Capabilities: Generative AI) sold as-a-service, used to drive **AML.T0052** phishing, **AML.T0102** malicious-command generation, and BEC fraud. Their real significance is *lowering the skill floor* for cybercrime.

6.1 WormGPT

The archetype. First announced as in-development on Hack Forums in March 2023 by a threat actor using the alias “**Last**”, and publicly released around June 2023, marketed as “the biggest enemy of ChatGPT” and a blackhat alternative. Security firm **SlashNext** analysed it in July 2023.

- **Base model:** reportedly **GPT-J 6B** — the open-source 6-billion-parameter model released by EleutherAI in 2021 — fine-tuned by the creator on malware code, exploit write-ups, and phishing material.
- **Primary use:** business-email-compromise (BEC) and phishing. SlashNext had it generate a BEC lure and described the result as remarkably persuasive.
- **Pricing:** reported around €60–€100/month or €550/year, with a private setup around €5,000.
- **Shutdown:** the original was shut down on **8 August 2023** after investigative journalist Brian Krebs identified the operator, drawing intense media attention.
- **Afterlife:** “WormGPT” became a *brand*. Researchers at Cato Networks (2025) uncovered new variants that were in fact jailbroken wrappers around commercial models (xAI’s Grok and Mistral’s Mixtral), and a later “WormGPT 4” was advertised with tiered pricing.

6.2 FraudGPT

Surfaced within days of WormGPT, advertised on the dark web and Telegram from around 22–25 July 2023 by an actor using the handle “**CanadianKingpin12**” (a known carding/fraud vendor). Documented by **Netenrich** and SlashNext.

- **Positioning:** an all-in-one offensive toolkit “with no boundaries.” Advertised capabilities: writing spear-phishing and BEC emails, creating undetectable malware, generating phishing/scam pages, finding vulnerabilities and leaks, carding, and hacking tutorials. The seller claimed the model was refreshed every one to two weeks and swapped between underlying models.
- **Pricing:** subscription — roughly \$200/month, ~\$1,000 for six months, up to \$1,700/year; the seller claimed 3,000+ sales.
- **Distribution shift:** the actor moved promotion to Telegram after clear-net forum threads were removed, and later teased related tools (“DarkBERT”, “DarkBARD”). Promotional channels and posts later disappeared.

6.3 XXXGPT

Announced on the **XSS.is** forum around late July 2023 (dark-web monitor Falcon Feeds flagged it on 31 July), pitched as a “revolutionary service offering personalized bot AI customization...with no censorship or restrictions,” reportedly backed by a team of five.

- **Claimed focus:** malicious *code* rather than just email — botnets, remote-access trojans (RATs), keyloggers, infostealers, ATM-malware kits, and cryptostealers.
- **Pricing:** reported around \$90 for a one-month license.
- **Credibility:** heavily disputed. Trend Micro noted there was essentially no follow-up posting, no user feedback, and no testimonials; Sophos found a tester unable to get the demo to produce malware and observed censored responses — consistent with a possible jailbroken-prompt wrapper or a scam. Its current status is unclear. A useful reminder that underground “capability” claims are marketing until verified.

Tool	First seen	Sold via	Reported base / nature	Headline use	Reported pricing
WormGPT	Mar–Jun 2023	Hack Forums	GPT-J 6B, fine-tuned (credible)	BEC / phishing	~€60–100/mo; ~€5k private
FraudGPT	Jul 2023	Dark web + Telegram	Undisclosed, rotated models	All-in-one fraud & malware	~\$200/mo – \$1,700/yr
XXXGPT	Jul 2023	XSS.is forum	Disputed; possible wrapper/scam	Malware code: botnets, RATs, keyloggers	~\$90/mo

Sources for Section 6: SlashNext, Netenrich, Cato Networks (Cato CTRL), Trend Micro, Sophos X-Ops, Outpost24, Krebs on Security, and Palo Alto Unit 42 reporting, 2023–2026. Figures are as reported by those researchers; underground pricing and capability claims are self-reported by sellers and vary between sources.

■ DEFENDER TAKEAWAY

You cannot block these tools at the source, and their output is designed to look legitimate. Defence shifts left and right of the model: **reduce believable pretext** (limit public info, brand-impersonation monitoring), **harden the human layer** against better phishing (out-of-band verification for payments, ATLAS AML.M0018 user training), and **detect on behaviour, not content** — the malware and fraud still have to touch your endpoints, identity, and payment rails, where conventional ATT&CK-aligned detection and controls still apply.

Using these notes

This is a study and reference document, written for defensive understanding. The technique catalogue is condensed from the published [dataset](#) (Apache-2.0, © 2021–2026 The MITRE Corporation, atlas.mitre.org) and [ATT&CK Enterprise](#). For authoritative, current detail — full technique text, sub-techniques, mitigations, and the 73 case studies — work from the live matrices, since both frameworks update monthly.

Baselines as of 28 September 2026: ATLAS 16 tactics / 208 techniques / 40 mitigations / 73 case studies (23 real-world incidents). ATT&CK Enterprise 15 tactics / 222 techniques — note the v19 Defense Evasion → Stealth + Defense Impairment split. Cross-check anything you operationalise against the source.